



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Employee Skill-Gap Analysis and Training Recommendation System

A PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA1016 – Software Engineering

to the award of the degree of
BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY
Submitted by

Rashmi Naurene (192521357)

Under the Supervision of
Dr. Anita Davamani K

JULY 2026

Table of Contents

S.NO	Title	Page Number
1.	Understanding the problem statement	4
1.1	What the actual problem is	
1.2	Objectives	
1.3	Scope	
2.	Stakeholders and Challenges	6
2.1	Who is actually in the room	
2.2	Two roles that matter the most	
2.3	Where the current approach breaks down	
3.	Analysis of the current approach	8
3.1	How this is done today	
3.2	To be fair to the old way	
3.3	Where it falls apart	
3.4	The three gaps	
4.	Proposed Software solution	10
4.1	Modules	
4.2	Architecture	
4.3	Functional requirements	
4.4	Non-Functional requirements	
4.5	Principles	
4.6	Scenario walkthrough	
5.	Justification of the chosen methodology	14
5.1	Agile, run as scrum	
5.2	The actual reasons	
5.3	Against Waterfall	

5.4	In practice	
6.	Expected Outcomes and Limitations	16
6.1	What gets better	
6.2	Mapping Fixes to Problems	
6.3	Where this can go wrong	
6.4	Where this could go next	

1. Understanding the Problem Statement

1.1 What the Problem Actually Is

Picture a mid-sized IT services company six months after it wins a contract requiring cloud-security expertise. HR knows, in a vague way, that "some people need upskilling." What it does not know — because nobody has ever measured it — is which fifty of its four hundred engineers are closest to job-ready, which are three courses away, and which are nowhere near it. The annual appraisal cycle will eventually surface some of this, but the contract deadline will not wait for the annual appraisal cycle. This is the gap the problem statement is really describing: organisations feel skill shortages long before they can name them precisely, because the tools they have — spreadsheets, PDF certificates, a manager's memory of who worked on what — were never built to answer "who can do X, and who is closest to being able to?"

Stripped of the HR jargon, Problem Statement 27 asks for three things chained together: a way to know what an employee can currently do, a way to know what their role requires them to do, and a way to automatically close the distance between the two with the right training, at the right time, tracked to completion. The word "system" in the title is doing real work here — the brief is not asking for a better spreadsheet template, it is asking for software that treats skill data the way an inventory system treats stock: continuously updated, queryable, and tied to an action (reorder, in this case retrain) the moment a shortfall is detected.

Concretely, the system needs to pull from four kinds of evidence — performance reviews, certifications, skill assessments, and project experience — reconcile them into one profile per employee, compare that profile against a role's requirements, and then behave less like a report generator and more like a coach: recommend, remind, and track until the gap closes.

1.2 Objectives

Rather than list the brief's objectives in the order they were given, it helps to sort them by what question each one is actually trying to answer.

"What can this person do, right now?"

Build the capability to evaluate employee competencies from multiple evidence sources, and compare that evaluation against what the employee's job role actually requires.

"What should they do next?"

Recommend training that is personalised to the specific gap found, not a generic catalogue push — and make sure the learning path respects the fact that a backend developer's competency framework has nothing in common with a finance analyst's.

"Did they actually do it?"

Track completion and certification progress over time, and chase people who have stalled, automatically, rather than relying on a manager to remember to follow up.

"Is the whole organisation getting better?"

Give HR and leadership a dashboard view that turns thousands of individual training records into a handful of decisions worth making — where to invest budget, which department is falling behind, which competency framework needs updating.

These four questions are not independent. Answering the third badly (poor tracking) quietly undermines the first (the next cycle's competency evaluation will be built on incomplete data). That interdependence is why the system has to be a single connected platform rather than four separate tools bolted together — a point that becomes important again in Section 4, when the modules are designed.

1.3 Scope

A system like this can sprawl. "Skills" touches recruitment, appraisals, succession planning, even payroll if you let it. The scope below is drawn deliberately narrow, and the exclusions matter as much as the inclusions.

What the system covers

- Competency evaluation for people already employed — not candidates, not applicants.
- Gap computation at three levels: individual, team, and department, since a manager and an HR director need answers at different resolutions.
- A recommendation layer that turns a detected gap into a specific, ranked list of training content.
- Dashboards split by audience — an employee's view is not a manager's view is not HR's view.
- Reminders and progress tracking, so that recommending training is not where the system's responsibility ends.
- Role-based access, since performance-linked data is sensitive by nature.

What it deliberately does not cover — for now

Recruitment and pre-hire assessment are excluded outright: this system evaluates people the organisation already has, not candidates it might hire. Compensation and promotion decisions are excluded too — the system surfaces gap data that a manager might use in a promotion conversation, but it does not make compensation decisions itself, and conflating the two would make employees far less honest in self-assessments. Full LMS content authoring is also out of scope; the platform recommends and links to training rather than building a course-hosting engine from scratch, which would be reinventing infrastructure that mature LMS platforms already provide well. Finally, forward-looking labour-market forecasting — predicting what skills the industry will need in three years — is left for a future release (Section 6.4); the first version is deliberately built to solve today's gap, not next decade's.

2. Stakeholders and Challenges

2.1 Who Is Actually in the Room

It is tempting to treat "stakeholders" as a formality — a table to fill in before getting to the interesting design work. But for this particular system the stakeholder map is the design work, because the same underlying data (an employee's skill profile) is read by six different audiences with six different levels of trust, six different needs for detail, and at least one outright conflict of interest between them (an employee wants training to look good; a manager sometimes wants gap data to justify a resourcing decision the employee might not welcome). Getting the roles right up front avoids building one dashboard and discovering later that nobody it was built for actually wanted that view.

Stakeholder	Description	Roles & Responsibilities
Employee	The person being assessed.	Feeds the system data (self-assessments, project logs); consumes its output (personal gap report, assigned path); is ultimately the one who has to act on it.
Reporting Manager	Direct supervisor.	Validates or contests the system's read on an employee's skills; watches team-level coverage; uses gap data in one-on-ones.
HR / L&D Administrator	Owens the framework the whole system is measured against.	Defines what "competent" means for each role; curates the training catalogue; is the person whose configuration mistakes propagate to every employee below them.
Senior Leadership	Budget holder, not a daily user.	Reads aggregate dashboards to decide where training spend goes and whether the organisation can staff its next contract.
IT / System Administrator	Keeps the lights on.	Access control, integrations with HRIS and LMS, uptime, backups — invisible when done right, very visible when not.
Training Providers (internal trainers, Coursera, NPTEL, certification bodies)	External to the org but internal to the workflow.	Supply the course metadata the recommendation engine actually matches against, and issue the completion signals that close the loop.

2.2 A Closer Look at Two Roles That Matter Most

Two stakeholders deserve more attention than the table above gives them, because the entire system stands or falls on their behaviour. The first is the HR/L&D administrator, who defines the competency framework each employee is measured against. Every gap score the system ever produces is only as good as this one person's (or team's) judgement about what a role requires — get the framework wrong, and the system will confidently and consistently mis-recommend training for every employee in that role.

There is no algorithmic fix for a badly specified framework; it is a human accuracy problem wearing a software interface.

The second is the employee themselves. Self-reported project experience is one of the four data sources feeding the gap engine, and self-reporting is inherently gameable — an employee under pressure to look competent has an incentive to overstate involvement in a project. The system's design has to acknowledge this rather than pretend it away, which is part of why manager validation (not just employee self-assessment) sits in the data pipeline in Section 4.

2.3 Where the Current Way of Doing Things Breaks Down for Each Group

Employees are, in practice, flying blind. Nobody tells them, in specific terms, what separates them from the next level of their role — they get a vague "work on your communication skills" comment in an annual review and are left to guess what that means in practice. The training they are pointed toward is whatever is in this quarter's catalogue, not what they specifically lack, so completion is low because relevance is low.

Managers are stuck making staffing calls from memory. Ask a manager of twelve people which three could take over a Kubernetes migration tomorrow, and most will guess based on who mentioned it in a hallway conversation, not on any recorded evidence. That is a real business risk hiding behind an HR process.

HR is doing manual data reconciliation that does not scale. Cross-referencing a performance review spreadsheet against a certifications inbox against a project-tracking tool, by hand, for a few hundred employees, is tedious; for a few thousand, it is simply not done, which is why most organisations' skill data is stale the moment it is collected. This is not a hypothetical inefficiency — industry research puts the number of companies reporting a meaningful skills gap at roughly 87 percent, which suggests the underlying problem this brief describes is close to universal, not specific to one company's bad habits.

3. Analysis of the Current Approach

3.1 How This Is Actually Done Today

Walk through a typical cycle. Once a year, an employee opens a form — sometimes a proper HRIS module, sometimes literally a shared spreadsheet — and rates themselves against a list of competencies that was probably written two role-redesigns ago. Their manager fills out a separate form, often without seeing the employee's version until a calibration meeting weeks later. Certificates get emailed to HR as PDF attachments and, if the organisation is disciplined, get logged in yet another spreadsheet; if not, they live in an inbox and are functionally invisible to anyone doing workforce planning. Project experience — arguably the strongest signal of what someone can actually do — is rarely captured at all in this process, because it lives inside Jira or a similar tool that HR has no access to and no reason to query.

At the end of this cycle, someone — usually the manager, sometimes HR — decides what training the employee should take. That decision is made by browsing whatever the training catalogue happens to contain that quarter, not by any systematic comparison against a documented gap. It is closer to picking a course off a shelf than prescribing a remedy.

3.2 To Be Fair to the Old Way

It is worth resisting the temptation to describe the manual process as purely broken, because it does get a few things right that a naive automated replacement could easily lose. A manager's judgement can absorb context an algorithm cannot — knowing that an employee's certification lapsed because of a family emergency, not a skill decline, is exactly the kind of nuance a raw data pipeline will flatten unless it is deliberately designed to leave room for override. The process also requires almost no infrastructure, which means even organisations with immature IT capability can run some version of it. And because it forces a manager and employee into a room together once a year, it produces a relationship-based understanding of career goals that a dashboard, however well designed, will not replicate on its own.

3.3 Where It Actually Falls Apart

The deeper problem is not that any single step is bad — it is that the process is annual in a world where technology shifts monthly. A cloud-migration project that lands in March creates a skill requirement the organisation will not formally notice until the next appraisal cycle, by which point the project is finished, badly, by people who were not ready for it. Layered on top of that timing problem is a data problem: the four sources of evidence (reviews, certifications, assessments, project logs) sit in at least three unconnected systems, so "what does this employee actually know" is a question nobody in the organisation can answer without a multi-day manual investigation.

Even where data exists, it is inconsistently calibrated — one manager's "4 out of 5" is another manager's "2 out of 5" for the identical underlying skill level, which makes any cross-team comparison close to meaningless. Training assignment, as noted above, is catalogue-driven rather than gap-driven, so completion rates suffer because employees correctly sense the training was not really for them. And once training is assigned, there is no follow-up mechanism at all — no reminder, no nudge, no visible

consequence for simply not finishing it, which is precisely the kind of thing software is good at and manual process is bad at.

None of this scales. A department of thirty people can survive on manager memory and a shared spreadsheet. An enterprise of four thousand cannot, and yet most enterprises are still running exactly this process at that scale, which is a large part of why the problem statement exists in the first place.

3.4 The Three Gaps That Actually Need Fixing

Pulling the threads above together, there are really only three distinct problems to solve, and they map cleanly onto the modules proposed in Section 4. There is a data problem — the evidence exists but is scattered across systems that do not talk to each other. There is an analysis problem — even where the data is available, nobody is systematically comparing it against an explicit standard. And there is a follow-through problem — even a correctly identified gap does nothing if the resulting training is never tracked to completion. A system that solves only the first two would still fail, because it would stop at diagnosis without treatment; a system that solves only the third would be blindly recommending training without knowing what is actually needed. All three have to be solved together, which is the case made in the next section.

4. Proposed Software Solution

4.1 The Shape of the Solution

The design choice that matters most here is refusing to treat this as a report-generation tool. A system that produces a gap report and stops is really just a fancier spreadsheet. What makes this a system, in the software-engineering sense, is that it closes a loop: data comes in, a gap is computed, a recommendation goes out, progress is tracked, and that tracking data becomes part of the input for the next cycle. Eight modules do this work between them, and the table below is deliberately written as "what it does," not "what it's called," because in a viva you will get more credit for being able to explain the function than for reciting the name.

Module	Description
Data Ingestion & Integration	Pulls in and normalises performance-review scores, certifications, project-experience records, and assessment results from wherever they currently live (HRIS, LMS, project-tracking tools).
Competency Framework Management	Where HR defines, versions, and edits what "competent" means for each role and department — the yardstick everything else is measured against.
Skill-Gap Analysis Engine	Compares an employee's assembled profile against their role's framework and produces a per-skill deviation score.
Training Recommendation Engine	Turns each flagged gap into a ranked shortlist of actual courses, drawing on internal content and external catalogues like Coursera or NPTEL.
Learning Path & Progress Tracking	Sequences recommended courses into a coherent path and tracks how far through it an employee actually is.
Notification & Engagement	Chases people — politely — about pending, upcoming, and overdue training, and flags stalled progress to managers.
Dashboard & Reporting	One engine, three very different views: personal gap report for the employee, team heatmap for the manager, organisation-wide trend lines for HR and leadership.
Access Control & Security	Makes sure the sensitive parts of all the above are only visible to people who are supposed to see them, and logs who looked at what.

4.2 How the Pieces Fit Together

There are four layers here, and each earns its place for a specific reason rather than because "layered architecture" is the textbook default. The presentation layer is a single application that renders three different dashboards depending on who is logged in — an employee, a manager, or an HR administrator see the same underlying data sliced completely differently, which is cheaper to build and maintain than three separate applications would be.

Below that sits the application layer, where each module from Section 4.1 runs as an independent service talking to the others through defined APIs. This is the layer that actually earns the word "modular": the recommendation engine can be rewritten from scratch — say, moving from keyword matching to something that understands semantic similarity between skill descriptions — without anyone having to touch the notification service or the dashboard code, because neither of those depends on how the recommendation engine works internally, only on the data it returns.

The data layer holds the structured records — employee profiles, competency frameworks, the training catalogue, progress history — in a relational database, with room to add a search or vector index later if the recommendation engine grows sophisticated enough to need one. And the integration layer is the honest acknowledgement that this system does not live alone: it has to talk to whatever HRIS and LMS the organisation already runs, or it will never get adopted, no matter how good the gap-analysis logic is.

4.3 Functional Requirements

Ten requirements, each traceable back to an objective in Section 1.2 or a module in Section 4.1 — deliberately kept to what the system must do, with the how left to Section 4.2 and the how well left to Section 4.4.

ID	Requirement
FR-1	HR administrators can define and edit competency frameworks per job role and department.
FR-2	The system imports performance, certification, and project-experience data from connected sources.
FR-3	A per-employee skill-gap score is computed by comparing profile against role framework.
FR-4	Each identified gap produces a ranked list of recommended training resources.
FR-5	Employees can enrol in recommended training and have their completion tracked automatically.
FR-6	Automated notifications go out for assigned, upcoming, and overdue training.
FR-7	Managers get a team-level dashboard of skill coverage and training progress.
FR-8	HR and leadership can export organisation-wide reports (PDF/CSV).
FR-9	Access is role-based, distinguishing employee, manager, and administrator permissions.
FR-10	A historical log of each employee's gap scores over time is retained, to show growth, not just a snapshot.

4.4 Non-Functional Requirements

This is the section that turns "modularity, scalability, maintainability, reliability, usability, security" from six words on a rubric into six specific, testable claims about the system. Each row below states what would actually have to be true for the requirement to be considered met — not just the principle's definition.

Quality Attribute	Requirement Description
Scalability	Should hold up whether it is serving one department of thirty people or a four-thousand-person enterprise, without needing a redesign in between — the application layer scales horizontally, the data layer can be partitioned by department or region.
Reliability	Two identical inputs should always produce the same gap score. Malformed or incomplete records get caught at ingestion rather than silently corrupting a downstream calculation.
Usability	A gap report or heatmap should be readable by someone who has never used the system before and has no analytics background — that is a harder bar than "the data is technically all there."
Maintainability	Swapping the recommendation engine's underlying algorithm — say, from keyword matching to something semantic — should not require touching the notification module or the dashboard. The module boundary is the maintenance boundary.
Security	Encryption at rest and in transit, role-based access control, and an audit trail of who accessed or edited what — this is performance-linked personal data, and it should be treated with the same seriousness as any other sensitive HR record.
Availability	Employees and managers will hit the dashboard at odd hours across time zones, so the target here is realistically 99.5%+ uptime on the core services rather than best-effort.
Interoperability	Talks to existing HRIS and LMS systems through defined APIs rather than asking the organisation to abandon tools it has already invested in.

4.5 Where the Six Principles Actually Show Up in the Design

It is one thing to list modularity, scalability, maintainability, reliability, usability, and security as requirements; it is another to point at the design and say exactly where each one lives. Modularity is not a slogan here — it is the reason the eight modules in Section 4.1 exist as separate services rather than one large program, and it is the same reasoning that shows up in real deployed systems in this space: an academic implementation called SkillSync, for instance, structures resume evaluation, gap detection, and recommendation as genuinely separate components precisely so each can be improved independently.

Scalability follows directly from the layered architecture in 4.2 — the application layer can add service instances as user load grows without touching the data layer, and the data layer itself can be partitioned by department if a single database instance ever becomes a bottleneck. This is not a hypothetical concern: commercial platforms operating at enterprise scale in this exact space rely on a single unified skills data model applied across many use cases, rather than duplicating data stores per feature, specifically to keep scaling tractable.

Maintainability shows up as a direct consequence of the module boundaries — because FR-1 requires versioned competency frameworks, a historical gap score computed under an older framework version stays interpretable even after HR updates the framework, which is the kind of small design decision that prevents a maintenance headache two years down the line. Reliability is handled at the point data enters the system: validation at ingestion means a malformed certification record gets rejected there rather than

quietly producing a wrong gap score three steps downstream, and the historical logging in FR-10 means any suspicious score can be traced back to its source.

Usability is the reason the presentation layer renders three different dashboards rather than one dense, do-everything screen — an employee does not need, and should not have to parse, the same interface an HR administrator uses to configure frameworks. And security is treated as load-bearing rather than a checkbox, precisely because this system centralises performance-linked personal data that did not use to live in one place; role-based access control and audit logging (Section 4.4) exist because the cost of getting this wrong is not a bug report, it is a trust and compliance problem.

4.6 Walking Through One Real Scenario

Take a data analyst who submits a quarterly self-assessment while their manager separately submits a review. In the background, a completed visualisation certification has already synced in from the LMS integration, and the technology stack from their most recent project has been logged automatically from the project-tracking integration. The ingestion module folds all four inputs into one profile. The gap engine pulls the "Data Analyst — Intermediate" framework and finds one clear shortfall: the employee's demonstrated proficiency in a specific visualisation tool sits below what the role expects, even though everything else checks out.

The recommendation engine queries the training catalogue for content tagged against exactly that tool, ranks the results by relevance and estimated time investment, and hands the employee a short, specific learning path rather than a generic course list — the same pattern used by comparable industry tools, which auto-generate links to structured course platforms the moment a specific missing skill is detected, rather than pointing at a whole catalogue. Reminders go out on a schedule; as modules get completed, the progress tracker updates; and by the time the next quarterly cycle rolls around, this employee's profile already reflects the closed gap, feeding a more accurate analysis next time round. That last part — the loop actually closing — is the entire point of building a system instead of a report.

5. Justification of the Chosen Methodology

5.1 Agile, Run as Scrum

Scrum is the recommendation here, not because it is the default answer for every student project, but because two specific traits of this system make it a genuinely better fit than a plan-driven alternative: the requirements are not fully knowable up front, and the architecture is already modular enough to be built and demonstrated in independent pieces.

5.2 The Actual Reasons, Not the Generic Ones

Competency frameworks are a moving target. Nobody — not HR, not the development team — can specify on day one exactly how gap scores should be weighted or exactly what a manager's dashboard needs to show, because those answers only become clear once real users see a first version and react to it. That is precisely the kind of uncertainty Agile's short feedback loops are built to absorb, and precisely the kind of uncertainty a Waterfall sign-off process handles badly, because by the time users see the finished product, changing course is expensive.

The modular architecture from Section 4.1 also happens to line up unusually well with sprint-based delivery. Data ingestion can be built, demoed, and refined as one increment; the gap-analysis engine as another; the recommendation engine as a third — each a working piece of software a stakeholder can react to, rather than one monolithic release nobody sees until the end.

Integration risk is the third factor. Nobody knows exactly how clean the organisation's existing HRIS or LMS data will turn out to be until the team actually tries to connect to it, and that is a discovery best made early, in a sprint, rather than during final testing under Waterfall, when there is no time left to adapt.

And because this system directly shapes how people's performance gets perceived, continuous stakeholder involvement is not a nice-to-have — it is what keeps HR (and, indirectly, employees) trusting the tool as it is being built, rather than being handed a finished system they had no chance to react to along the way.

5.3 Against the Obvious Alternative

The natural comparison is Waterfall, since it is the model most students default to when asked to justify a choice. The table below is deliberately built around dimensions specific to this project, not a generic Agile-vs-Waterfall textbook comparison.

Dimension	Waterfall	Agile / Scrum
Requirement changes	Expensive once a phase is signed off	Expected every sprint, built into the process
Stakeholder involvement	Front-loaded and back-loaded — heavy at the start and the end, quiet in between	Continuous, through sprint reviews and demos

Dimension	Waterfall	Agile / Scrum
Delivery style	One large release at the very end	Working software every 2–4 weeks
Fit for this specific project	Weak — the framework and recommendation logic will keep shifting as HR actually sees results and reacts to them	Strong — early modules can be validated with real HR users while later ones are still being built

5.4 What This Looks Like in Practice

A realistic sprint sequence would put data ingestion and the competency-framework module first, since everything else depends on them — there is no point building a recommendation engine against data that does not exist yet. The gap-analysis engine follows, then the recommendation engine, then the dashboard and notification layers last, once there is something meaningful to display and something worth reminding people about. Each sprint ends with a demo to HR stakeholders and a retrospective, which is where the riskiest assumptions — data quality from the HRIS integration, whether the competency-framework model actually matches how HR thinks about roles — get tested while they are still cheap to fix.

This same rhythm carries past initial launch. Competency frameworks will need updating as roles and technologies shift, and training catalogues will need refreshing — that is not a separate "maintenance phase" requiring a different process, it is simply the backlog continuing to run through the same sprint cadence indefinitely.

6. Expected Outcomes and Limitations

6.1 What Actually Gets Better

The most immediate benefit is that gap identification stops being a once-a-year guess and becomes a continuously updated, multi-source read on where each employee actually stands. That, in turn, makes training recommendations specific rather than generic — and specific training gets finished at higher rates than catalogue-browsing ever does, because employees can tell the difference between a course chosen for them and a course chosen at them.

Managers gain something they currently do not have at all: a real-time view of what their team can do, which turns staffing decisions from a guess based on hallway conversations into something backed by evidence. HR gains the same thing at organisation scale — a defensible basis for training budget requests, instead of an intuition. And because notifications and visible progress tracking exist where they did not before, the completion-rate problem identified in Section 3.3 gets addressed structurally, not just hoped away.

6.2 Mapping Fixes to the Original Problems

Each of the three structural gaps from Section 3.4 has a corresponding fix built into the design. Fragmented data across HRIS, LMS, and project tools gets consolidated by the ingestion module into one profile per employee. Subjective, ad-hoc comparison gets replaced by the gap-analysis engine's systematic comparison against an explicit framework. And the follow-through failure — gaps identified but never actually closed — gets addressed by the combination of recommendation, tracking, and notification working together, so that diagnosis is no longer where the process stops.

6.3 Where This Can Still Go Wrong

None of the above works if the underlying data is bad, and that is worth stating plainly rather than glossing over: a gap-analysis engine is only as trustworthy as the performance reviews, certifications, and project logs feeding it, and no amount of clever analysis fixes garbage input. Competency frameworks carry a similar risk on the HR side — if nobody keeps them updated as roles and technologies evolve, the whole system will keep confidently recommending training against an outdated standard, which is arguably worse than having no system at all, because it looks authoritative.

Integration with existing HRIS and LMS systems is likely to be the single most time-consuming part of actually building this, particularly if those systems use inconsistent data formats or lack modern APIs — this is a genuine implementation risk, not a formality to mention and move past. There is also a trust problem worth naming honestly: because this system shapes how performance is perceived, employees and managers who do not understand how a gap score was calculated have every reason to be skeptical of it, and skepticism left unaddressed turns into disengagement. Centralising sensitive performance data in one place also raises the stakes of any security failure, which is exactly why security was treated as a core requirement in Section 4.4 rather than an afterthought. And a first version built on simple rule-based

matching between gaps and course content will inevitably miss subtler or more novel skill matches that a more sophisticated approach could catch — a limitation, not a flaw, and one with a clear next step below.

6.4 Where This Could Go Next

- **Smarter gap detection.** Moving from rule-based comparison toward transformer-based semantic matching between job descriptions, project experience, and skill taxonomies — academic implementations in this space already show this improves match precision over simple keyword comparison.
- **Forward-looking recommendations.** Pulling in external labour-market and industry-trend data so the system flags not just today's gap against today's framework, but skills likely to matter in eighteen months — a capability increasingly offered by more advanced commercial skills-intelligence platforms.
- **Internal mentor matching.** Recommending a colleague who already has the missing skill, not just a course — turning the organisation's own skills graph into a resource, not only an external catalogue.
- **Tighter LMS integration.** Two-way syncing so quiz scores and time-on-task feed back into the competency profile automatically, without anyone re-entering data by hand.
- **Behavioural and soft-skill coverage.** Extending the framework beyond technical competencies — a gap most comparable commercial tools still leave largely unaddressed.
- **Explaining the score, not just giving it.** As the analysis gets more sophisticated, pairing every gap and recommendation with a plain-language explanation of why it was generated — directly answering the trust problem raised above, rather than asking employees to take the algorithm's word for it.